

You can take a video of a few falling objects and convert them into frames. So you may get about 30 frames per second. Identifying an object in a video frame will not be very precise but the errors in measurement are an inherent part of research. The model you wish to verify is: $y = \frac{1}{2} g t^2 + v_0 t + y_0$

You have a list of the object's position every 1/30th of a second. You can fit the values to a second-degree polynomial least-square fit algorithm. Here is the code you would use:

```
import numpy as np
values = [195.7,193.51,190.59,185.84,180.73,173.43,
165.76,157.36,147.87,136.92,124.87]
y = np.array(values)
t = np.linspace(0, 10.0/30, 11)
poly_coeff = np.polyfit(t, y, 2)
print poly_coeff
print 'g = %.2f'%poly_coeff[0]
```

You convert a Python list of 11 values into a Numpy array. The `linspace` method creates an equally spaced array, with the first value being 0 and the last (11th) value being 1/3 sec. The `polyfit` method will fit the data to a polynomial with the degree being 2 and return an array of three numbers. These three numbers are the coefficients of the polynomial starting with the highest power first. So, the first value is half of g . The above data gives g as -963 cm/sec^2 . The negative sign indicates the downward direction.

So, how good was the fit?

Before discussing that, let us look at the relationship between an array and a polynomial.

Polynomials and arrays

Any polynomial can be represented by a vector, just by keeping track of the coefficients. For example, $[1,0,-4]$ would represent $(x^2 - 4)$. Try the following examples:

```
>>> import numpy as np
>>> p2 = np.poly1d([1,0,-4])
>>> p2(0)
>>> p2([0,1,2])
```

You have created an object of the type polynomial (or simply a polynomial function) of degree 2. Now, you can use it like you would a function. The result for `p2(0)` would be -4 and for `p2([0,1,2])`, it would be the array $[-4, -3, 0]$.

The best way to see how well the polynomial you obtained fits the data is to visually see it through a plot.

Matplotlib

The `python-matplotlib` package must be installed.

You will need to plot the original data and the values computed from the polynomial. Hence, add the following

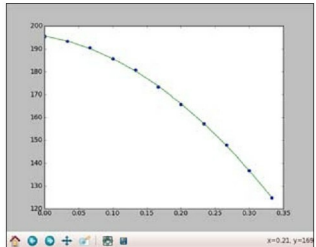


Figure 1: A Falling Object - Observed positions and the theoretical fit to a quadratic polynomial

code after obtaining the least square fit to your data:

```
import matplotlib.pyplot as plt
plt.plot(t, y, 'o')
plt.plot(t, np.poly1d(poly_coeff)(t), '-')
plt.show()
```

The first call to the plot function plots small circles at the actual pairs of data points (t, y) .

In the second call to the plot function, you are creating a polynomial function from the array `poly_coeff` and then computing the values of this function for each value of t . Finally, you are plotting these values against t as a line. The resulting fit for the above example is pretty good as can be seen in Figure 1.

The roots

Solving a polynomial is another common requirement. The roots of the polynomial that fit to the data would indicate when the object would be on the ground. Add the following lines to your code:

```
roots = np.roots(poly_coeff)
print 'Roots', roots
```

The output of the program, including the earlier code, would be:

```
[-481.72027972 -52.31748252 195.86951049]
g = -963.440559441
Roots [-0.69426607 0.58566055]
```

The meaningful root in this case is .585 sec. It may not be very useful in this context, but it does become valuable when dropping food packets or bombs from an aeroplane.